7 Marks

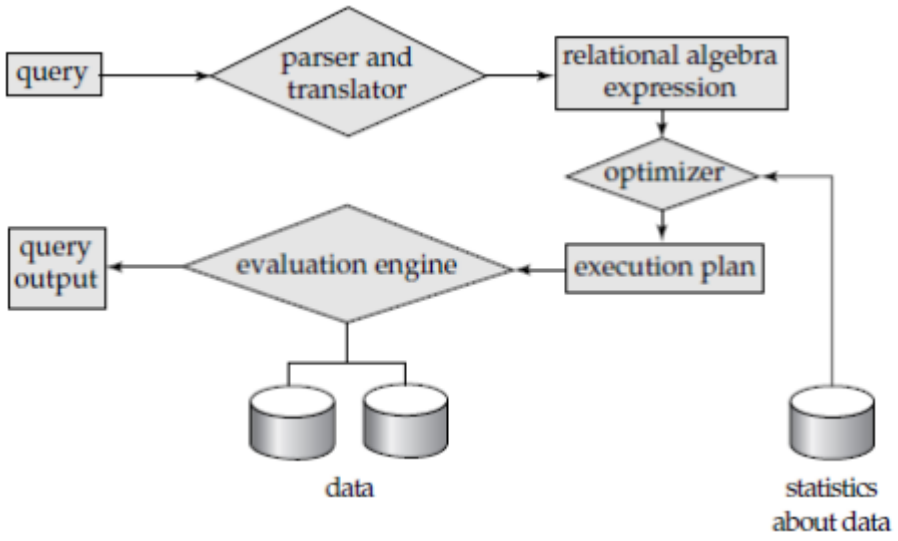
There are two transactions T and T'. T takes $100 rom account A to account B while T takes 10% of account A to account B. Property of T and T does not change when we do A+B. Create two serial executions for T and T'.

- **Transaction T:**
  - T transfers a fixed amount of $100 from Account A to Account B.
  - After the transaction, $100 is deducted from Account A and added to Account B.
- **Transaction T':**
  - T' transfers 10% of the balance in Account A to Account B.
  - After the transaction, 10% of Account A's balance is deducted from Account A and added to Account B.
- **Serial Execution 1:**
  1. **T executes first:**
     - Transaction T deducts $100 from Account A and adds it to Account B.
     - Account A is now reduced by $100, and Account B is increased by $100.
  2. **T' executes second:**
     - Transaction T' now transfers 10% of Account A's balance (after T) to Account B.
     - If Account A initially had $200, after T, it will have $100. So, T' will transfer 10% of $100, i.e., $10, from Account A to Account B.
     - Account A is reduced by $10, and Account B is increased by $10.
- **Serial Execution 2:**
  1. **T' executes first:**
     - Transaction T' first transfers 10% of Account A's balance (before T) to Account B.
     - If Account A initially had $200, T' will transfer 10% of $200, i.e., $20, from Account A to Account B.
     - Account A is reduced by $20, and Account B is increased by $20.
  2. **T executes second:**
     - Transaction T then deducts $100 from Account A (after T') and adds it to Account B.
     - Account A now has $180 (after T'), so Account A is reduced by $100, and Account B is increased by $100.
- **Note:** The property of commutativity in these transactions holds since both transactions only transfer amounts between two accounts and do not affect other operations or

dependencies. Therefore, the final state of the accounts is independent of the order of execution.

How does query processing work? Explain necessary steps in query processing with suitable diagram.

- Query processing involves several steps to translate a high-level query into an optimized execution plan that is run on a database system. These steps ensure that the query is executed efficiently to minimize resource usage (time, CPU, I/O).
- **Steps in Query Processing:**
  1. **Parsing and Translation:**
     - The SQL query is first parsed to check for syntactical correctness.
     - The query is translated into an intermediate representation, typically a relational algebra expression or a query tree.
  2. **Optimization:**
     - The query optimizer takes the intermediate representation and tries to find the most efficient way to execute the query.
     - It explores multiple execution plans, considering factors such as join methods, access paths, and indices.
     - The goal is to minimize the overall cost of the query, considering factors like I/O and CPU usage.
  3. **Execution Plan Generation:**
     - After optimization, the best execution plan is selected, which details the steps and order of operations (e.g., which indices to use, join methods).
  4. **Execution:**
     - The database management system (DBMS) executes the query based on the chosen plan.
     - The DBMS retrieves the required data from storage and applies operations such as joins, selections, and projections.
  5. **Result Return:**
     - Finally, the results of the query are returned to the user or application.

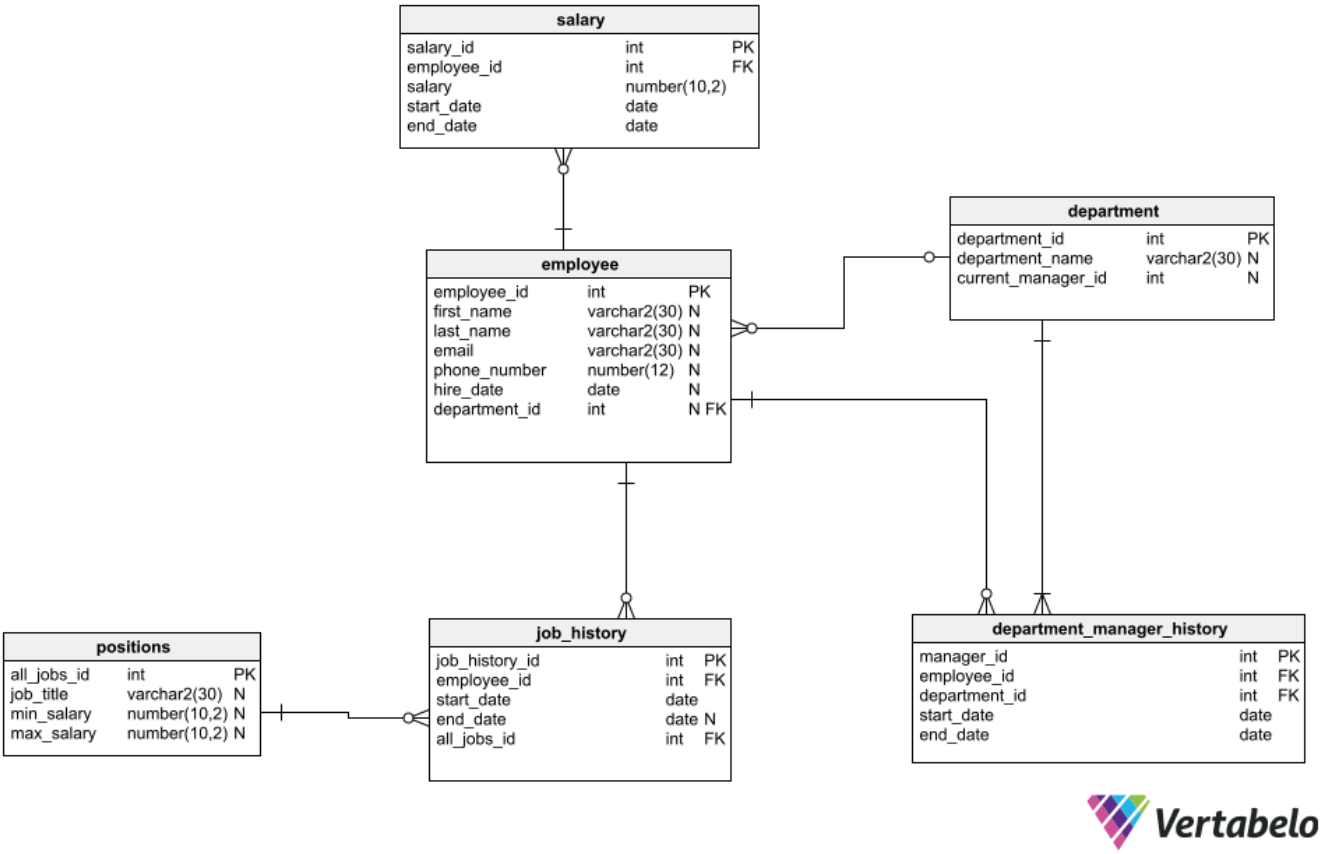


Explain Two-phase locking method with suitable example.

- **Two-phase Locking (2PL):** Two-phase locking is a concurrency control protocol used in databases to ensure serializability of transactions. It involves two distinct phases during the execution of a transaction:
  1. **Growing Phase:**
     - In this phase, a transaction can acquire locks but cannot release any.
     - The transaction keeps acquiring locks as needed, but it does not release any locks until the growing phase ends.
  2. **Shrinking Phase:**
     - Once the transaction releases a lock, it enters the shrinking phase.
     - During the shrinking phase, the transaction can only release locks but cannot acquire any new locks.
- The two-phase locking ensures that once a transaction starts releasing locks, it will not be able to interfere with other transactions, preventing issues like deadlocks and ensuring consistency.
- **Example:**
  - Let's consider two transactions, T1 and T2, executing on the same database with two items, A and B.
  - **Transaction T1:**
    1. Locks A in growing phase.
    2. Locks B in growing phase.
    3. Releases lock on A in shrinking phase.
    4. Releases lock on B in shrinking phase.
  - **Transaction T2:**
    1. Waits for T1 to release lock on A.

2. Once T1 releases A, T2 locks A.
3. T2 locks B.

Create an ER Model for Employee Management System.



Define functional dependency. Explain different functional dependencies with suitable rm example.

- Functional dependencies are a key concept in relational database design and are crucial for the process of **normalization**, which organizes data to reduce redundancy and ensure integrity.
- In a functional dependency, the value of one set of attributes in a database uniquely determines the value of another set of attributes. Understanding functional dependencies helps in structuring databases efficiently.
- **Definition**:
  - A functional dependency, denoted as **X** → **Y**, means that if two tuples (rows) in a relation (table) have the same values for attribute (s) X, they must also have the same values for attribute (s) Y.
- **Example**:
  - In a student table, if the student ID uniquely determines the student name, we say that "Student ID → Student Name."

- **Types :**
  - Trivial Functional Dependency :
    - A functional dependency is **trivial** if the dependent attribute is a subset of the determinant.
    - Notation : X → Y is trivial if Y is a subset of X.
    - Example : In the dependency `{Student ID, Name} → Student ID`, the left side (determinant) includes the right side, making it trivial.
  - Non-Trivial Functional Dependency:
    - **Definition**:
      - A functional dependency is **non-trivial** if the dependent attribute is not a subset of the determinant.
    - **Notation**:
      - X → Y is non-trivial if Y is not a subset of X.
    - **Example**:
      - In the dependency `Student ID → Name`, the name is not part of the student ID, so it's non-trivial.
  - Fully Functional Dependency:
    - **Definition**:
      - A functional dependency is **fully functional** when every attribute in the determinant is necessary to determine the dependent attribute.
    - **Notation**:
      - X → Y is fully functional if removing any part of X means Y is no longer dependent on X.
    - **Example**:
      - `{Student ID, Course ID} → Grade`. Here, both the student ID and course ID are needed to determine the grade. If you remove either, you can't determine the grade.
  - Partial Functional Dependency:
    - **Definition**: A functional dependency is **partial** when only a part of the determinant is enough to determine the dependent attribute.
    - **Notation**: X → Y is partial if some attribute in X can be removed while still preserving the dependency.
    - **Example**: `{Student ID, Course ID} → Student Name`. In this case, `Student ID` alone can determine the `Student Name`, making the dependency partial.
  - Transitive Dependency:
    - **Definition**: A transitive dependency occurs when one attribute is indirectly dependent on another attribute through a third attribute.
    - **Notation**: X → Y and Y → Z imply X → Z.

        - **Example**: If `Student ID → Department` and `Department → Dean`, then `Student ID → Dean`. The dependency is transitive because the Student ID indirectly determines the Dean.
- Example:
    - Consider a relation **Student** with attributes: `StudentID, CourseID, Instructor` And the following functional dependencies:
        - **StudentID → Name**
        - **CourseID → Instructor**
- If we have a tuple like: `StudentID = 123, CourseID = "CS101", Instructor = "Dr. Smith"`
- The functional dependencies indicate:
    - **StudentID → Name**: The StudentID uniquely determines the student's Name.
    - **CourseID → Instructor**: The CourseID uniquely determines the Instructor for that course.

Why normal form is needed in database tables? Explain 1st 2nd 3rd and BCNF normal form with suitable example.

- Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It ensures that data is stored in such a way that the following issues are minimized:
    - **Data redundancy** (duplicate data)
    - **Update anomalies** (inconsistencies when updating data)
    - **Deletion anomalies** (loss of valuable data when deleting a record)
    - **Insertion anomalies** (problems with inserting new data)
- 1st Normal Form (1NF):
    - A relation is in **1NF** if it satisfies the following:
        - Each column contains atomic (indivisible) values.
        - Each column contains values of a single type.
        - The order in which data is stored does not matter.
    - Example :
        - `StudentID | Name | Courses` 1 | John | Math,Science `` 2 | Alice | English, History `
        - This violates 1NF because the `Courses` column contains multiple values. To make it 1NF, we must ensure that each course is listed separately:
            - `StudentID | Name | Course` 1 | John | Math `` 1 | John | Science `` 2 | Alice | English `` 2 | Alice | History`
- **2nd Normal Form (2NF):**

- A relation is in **2NF** if:
  - It is in **1NF**.
  - There is no partial dependency, i.e., no non-prime attribute is dependent on a part of a candidate key.
- **Example:**
  - Consider a relation where `StudentID, CourseID` is the primary key:
    - `Student_Course(StudentID, CourseID, InstructorName)`
  - Here, `InstructorName` depends on `CourseID`, not on the full key `(StudentID, CourseID)`. To achieve 2NF, we decompose the table:
    - `Student_Course(StudentID, CourseID) Course_Instructor(CourseID, InstructorName)`
  - Now, `InstructorName` depends only on `CourseID`, not on the composite key.

- **3rd Normal Form (3NF):**
  - A relation is in **3NF** if:
    - It is in **2NF**.
    - There is no transitive dependency, i.e., non-prime attributes should not depend on other non-prime attributes.
  - **Example:**
    - Consider a relation:
      - `Student_Course(StudentID, CourseID, InstructorName, InstructorPhone)`
    - Here, **InstructorPhone** depends on **InstructorName** (a transitive dependency). To achieve 3NF, we decompose the relation:
      - `Student_Course(StudentID, CourseID, InstructorName)` `Instructor(InstructorName, InstructorPhone)`

- **Boyce-Codd Normal Form (BCNF):**
  - A relation is in **BCNF** if:
    - It is in **3NF**.
    - For every functional dependency, the left-hand side (determinant) is a superkey.
  - **Example:**
    - Consider the relation:
      - `Student_Course(StudentID, CourseID, InstructorName)`
    - If **InstructorName** → **CourseID**, then **InstructorName** is a determinant, but it's not a superkey. To make it BCNF, we decompose the table into:
      - `Instructor_Course(InstructorName, CourseID)` `Student_Course(StudentID, CourseID)`
  - This decomposition eliminates the violation of BCNF, as now the determiners are superkeys.

- **4th Normal Form (4NF):**
  - A relation is in **4th Normal Form (4NF)** if:
    - It is in **Boyce-Codd Normal Form (BCNF)**.
    - It does not contain **multivalued dependencies**.
  - A multivalued dependency (MVD) occurs when one attribute (or set of attributes) determines a set of values for another attribute, and this relationship holds independently of other attributes in the table. This means an attribute can have multiple independent values, and these independent values need to be stored in a separate relation.
  - **Example:** Consider a relation **Employee** where an employee has multiple skills and multiple languages they speak:
    - Employee(EmpID, Skill, Language)
    - Here,
      - **EmpID** → **Skill** (an employee can have many skills),
      - **EmpID** → **Language** (an employee can speak many languages).
    - In this case, **EmpID** determines both **Skill** and **Language**, but these two sets are independent of each other. This creates a **multivalued dependency**.
    - To convert this to 4NF, we decompose the relation into two relations:
      - `Employee_Skills(EmpID, Skill)` `Employee_Languages(EmpID, Language)`
    - Now, each relation has no multivalued dependencies because **Skill** and **Language** are stored independently of each other.
- **5th Normal Form (5NF):**
  - A relation is in **5th Normal Form (5NF)**, also called **Project-Join Normal Form (PJNF)**, if:
    - It is in **4th Normal Form (4NF)**.
    - It cannot be decomposed into smaller relations without losing information (i.e., no join dependency exists that could be decomposed further).
  - A **join dependency** occurs when a relation can be decomposed into multiple smaller relations, and the original relation can be recovered by performing a **natural join** on those smaller relations.
  - **5NF Example:**
    - Consider a relation that stores the information about courses taught by professors and students enrolled in those courses:
      - `Course_Professor_Student (CourseID, ProfessorID, StudentID)`
    - Here, the functional dependencies might be:
      - **CourseID, ProfessorID** → **StudentID** (each course and professor have specific students),

        - **CourseID, StudentID → ProfessorID** (each student is assigned a specific professor for a course),
        - **ProfessorID, StudentID → CourseID** (each student and professor combination relates to a particular course).
    - In this case, there is a **join dependency**. The relation can be decomposed into three smaller relations:
        - `Course_Professor(CourseID, ProfessorID)` `Professor_Student(ProfessorID, StudentID)` `Course_Student(CourseID, StudentID)`

What is an Index? Explain primary indexing and secondary indexing with examples.

- **Index:** An **index** is a data structure that improves the speed of data retrieval operations on a database table at the cost of additional storage space. It works like an index in a book, where it points to specific data locations, enabling faster lookups and efficient searching.
- **Primary Indexing:** Primary indexing refers to creating an index on the **primary key** of a table. The primary key is a unique identifier for each record, and the primary index ensures that the records are organized in a sequential order based on this key.
- **Characteristics of Primary Index:**
    - It is always unique because the primary key is unique for each record.
    - It organizes the table based on the primary key.
    - The primary index can be either a **dense index** (where every record has an index entry) or a **sparse index** (where only some records are indexed).
- **Example:** Consider a table `Employee` with the attributes `EmpID` (primary key), `Name`, and `Salary`:
    - `EmpID | Name | Salary
    - `1001 | John | 5000`
    - `1002 | Alice | 6000`
    - `1003 | Bob | 5500`
- The primary index would be built on `EmpID`:
    - Index on EmpID:
    - `EmpID → Record location`
    - `1001 → Record 1`
    - `1002 → Record 2`
    - `1003 → Record 3`
- **Secondary Indexing:** Secondary indexing refers to creating an index on a non-primary key, or on columns that are not unique, to speed up searches based on those attributes. This index is used for efficient querying when searching by non-primary key attributes.

- **Characteristics of Secondary Index:**
  - It is not unique and can be created on non-key columns.
  - It may include a **pointer** to the location of the record or the index itself may have duplicates, pointing to multiple rows with the same value.
- **Example:**
  - Consider a `Student` table:
    - `StudentID | Name | Age`
    - `S101 | John | 22`
    - `S102 | Alice | 23`
    - `S103 | Bob | 22`
  - If we create a secondary index on `Age`:
    - `Index on Age:`
    - `Age → Record location`
    - `22 → Record 1, Record 3`
      - `23 → Record 2`

Explain B+-tree index with suitable example.

- **B+-Tree Index:** A **B+-tree** is a self-balancing tree data structure used in database systems and file systems for indexing. It is an extension of the **B-tree**, where all values are stored in the leaf nodes, and internal nodes only store keys. This provides efficient range queries and searches.
- **Properties of B+-tree:**
  - It is a balanced tree, meaning all leaf nodes are at the same level.
  - It supports efficient insertion, deletion, and search operations with a logarithmic time complexity (O(log n)).
  - It can handle a large number of keys and can be used in both primary and secondary indexing.
- **B+-Tree Structure:**
  - **Internal Nodes** contain keys (not data).
  - **Leaf Nodes** contain the actual data or pointers to the data.
- **Example:**
  - Consider a **B+-tree** with an order of 3, meaning each node can have a maximum of 3 children and 2 keys.
  - Let's index a dataset with the values: `15, 25, 35, 50, 55, 60, 75, 80, 85`.
  - The B+-tree would look like this:
    - --[35]--
      /
      [15, 25] [50, 60]

/ | \ / |
[15] [25] [35] [50] [55] [75, 80, 85]

Discuss the various file organization methods (Heap, Sequential, Indexed, and Hashed) in detail.

- **Heap File Organization:**
  - In **heap file organization**, records are stored in no particular order. When a new record is added, it is inserted into the next available space in the file, typically at the end.
  - **Advantages:**
    - Simple to implement and store data.
    - No overhead for maintaining order or indexes.
  - **Disadvantages:**
    - Searching for a specific record is slow, as a full table scan may be required.
    - Insertion and deletion can result in fragmentation, reducing efficiency over time.
  - **Example:** A table where records are just added in the order they arrive without any index or sorting:
    - `Record1 | Record2 | Record3 | Record4`
- **Sequential File Organization:**
  - In **sequential file organization**, records are stored in sorted order based on a key. When inserting a new record, it is placed at the correct position to maintain the order.
  - **Advantages:**
    - Efficient for sequential access or range queries.
    - Faster search operations using binary search, as the data is sorted.
  - **Disadvantages:**
    - Insertion and deletion operations can be slow because the data must be reorganized to maintain the order.
    - Requires extra space to maintain order during inserts.
  - **Example:** A sorted list of employees based on `EmpID`:
    - `EmpID=1 | EmpID=2 | EmpID=3 | EmpID=4`
- **Indexed File Organization:**
  - In **indexed file organization**, an index is created for a file, allowing fast access to records using the indexed attributes. An index file maps the search key (e.g., `EmpID`) to the location of the record in the data file.
  - **Advantages:**
    - Fast search and retrieval using the index.
    - Allows fast updates and deletions.

- **Disadvantages:**
  - Additional storage required for the index.
  - Maintaining the index can be complex, especially during insertions and deletions.
- Example :
  - `Data File: | Index:`
  - `EmpID=1, Name=John | EmpID=1 -> Position 0`
  - `EmpID=2, Name=Alice| EmpID=2 -> Position 1`
  - `EmpID=3, Name=Bob | EmpID=3 -> Position 2`

- **4. Hashed File Organization:**
  - In **hashed file organization**, a **hash function** is used to map the search key (e.g., `EmpID`) directly to a location (bucket) in the file. Each bucket stores one or more records.
  - **Advantages:**
    - Extremely fast for equality searches.
    - Hashing provides direct access to data, making lookups faster.
  - **Disadvantages:**
    - Inefficient for range queries, as records are not stored in a sorted order.
    - Hash collisions can occur, where multiple keys map to the same bucket.
  - **Example:**
    - If we use a simple hash function `Hash(EmpID) = EmpID % 10` for `EmpID=15`, we would map `15 % 10 = 5`, so the record with `EmpID=15` would be stored in bucket 5.
    - `Buckets:`
      `0 -> Record 10`
      `1 -> Record 11`
      `2 -> Record 12`
      `3 -> Record 13`
      `4 -> Record 14`
      `5 -> Record 15`

- **Summary:**
  - **Heap file organization** stores records with no specific order, leading to slower searches.
  - **Sequential file organization** sorts records and is efficient for range queries, but slow for updates.
  - **Indexed file organization** uses an index to improve retrieval speed, making it suitable for both exact searches and range queries.

- **Hashed file organization** uses a hash function for direct access, which is efficient for exact match searches but not for range queries.

3 Marks

Write the significance of ACID properties in a database.

- The ACID properties (Atomicity, Consistency, Isolation, Durability) are crucial for ensuring reliable transaction processing in a database:
  - **Atomicity**:
    - Ensures that all operations within a transaction are completed successfully. If one part fails, the entire transaction is rolled back, leaving the database in a consistent state.
  - **Consistency**:
    - Guarantees that a transaction will bring the database from one valid state to another, preserving data integrity. Any changes made during the transaction are compliant with the defined rules, constraints, and data integrity.
  - **Isolation**:
    - Ensures that concurrently executed transactions do not affect each other. The operations of one transaction are isolated from others, even if they occur simultaneously, ensuring that intermediate states are not visible to other transactions.
  - **Durability**:
    - Once a transaction is committed, it is permanently stored in the database, even in the event of a system crash. The changes made are not lost.
- These properties help in maintaining the integrity, reliability, and correctness of data in a multi-user, distributed database system.

How does locking work in SQL? Describe different types of locks.

- Locking in SQL is a mechanism used to prevent conflicts and ensure data consistency when multiple transactions try to access the same data concurrently. It prevents other transactions from modifying or reading data that is currently being modified by another transaction. There are several types of locks in SQL:
  - **Shared Lock (S-lock)**:
    - Allows a transaction to read data, but prevents other transactions from modifying it. Multiple transactions can hold shared locks on the same data item concurrently.
  - **Exclusive Lock (X-lock)**:

        - Prevents both read and write access to the data by other transactions. Only one transaction can hold an exclusive lock on a data item at a time.
    - **Intent Lock**:
        - A higher-level lock used to indicate that a transaction intends to acquire a specific type of lock (e.g., shared or exclusive) on a data item within a page or table. It helps in preventing conflicts when multiple transactions are working with different parts of the same resource.
    - **Update Lock (U-lock)**:
        - A lock that prevents other transactions from acquiring an exclusive lock on the data, but still allows shared locks. It's typically used in situations where a transaction is intending to update data, but not yet doing so.
    - **Table-Level Lock**:
        - A lock applied to the entire table, which prevents any other transaction from accessing any rows of the table until the lock is released.
- Locking ensures data integrity and consistency, particularly in concurrent transactions. However, improper use of locks can lead to issues like deadlocks, where two or more transactions are waiting for each other to release locks, causing them to hang.

Explain the steps involved in query processing from parsing to execution.

- Query processing in a database involves multiple steps to efficiently execute a query. The process can be broken down into the following stages:
    - **Parsing**: The SQL query is first parsed to check for syntax errors. The query is transformed into a parse tree, which represents the structure of the query and checks for any logical or syntactic errors.
    - **Translation**: The parse tree is then translated into a relational algebra expression or a query tree. This step converts the SQL query into a more usable form for the optimizer to process.
    - **Optimization**: The query optimizer evaluates different query execution plans. The optimizer looks at the possible ways to access the data, choose the most efficient plan, and reduce the execution time, considering factors such as available indexes and statistics.
    - **Execution Plan Generation**: Once the best execution plan is selected, an execution plan is generated, which outlines how the query will be executed (e.g., the use of indexes, joins, scans, etc.).
    - **Execution**: The final step involves executing the query using the chosen plan. The database engine fetches the data, performs the necessary operations (such as joins, aggregations, etc.), and returns the result to the user.

- These steps ensure that queries are executed in an efficient, optimized manner, providing accurate and timely results to users.

Why database design methodologies are needed? Write its significance.

- Database design methodologies are essential for creating efficient, scalable, and reliable databases. These methodologies provide systematic approaches to designing databases that meet both functional and performance requirements.
- **Significance of Database Design Methodologies:**
  - **Data Integrity**: Well-designed databases ensure that the data is accurate, consistent, and valid by enforcing rules like primary keys, foreign keys, and unique constraints.
  - **Efficiency**: Proper design minimizes data redundancy and ensures efficient data retrieval. It helps in creating optimal relationships between tables, reducing storage space, and improving query performance.
  - **Scalability**: Good design practices help in scaling the database to handle increasing volumes of data, ensuring it can support future growth.
  - **Data Redundancy Reduction**: Effective design minimizes duplicate data, which not only saves storage space but also ensures that changes made in one place do not cause inconsistencies.
  - **Maintainability**: With structured database design, the system becomes easier to maintain, troubleshoot, and extend without introducing problems or breaking functionality.
  - **Security**: Proper design ensures that data is accessible only to authorized users, preventing unauthorized access and ensuring confidentiality.
- Database design methodologies like ER modeling (Entity-Relationship) or normalization help in structuring the database logically, ensuring it supports business needs while being efficient and easy to manage.

What will be the output of following statement? `SELECT * FROM employee WHERE name LIKE "A%":`

- The SQL query will retrieve all rows from the `employee` table where the `name` column starts with the letter **"A"**.
- The `LIKE` operator is used for pattern matching. The `"A%"` pattern means:
  - `"A"`: The name must start with the letter "A".
  - `"%"`: The `%` is a wildcard that matches any sequence of characters (including no characters at all).
- Thus, the output will be all the rows from the `employee` table where the `name` starts with "A", such as "Alice", "Alex", "Adam", etc.

Explain SQL WHERE Clause with UPDATE Statement with example.

- The `WHERE` clause is used in SQL to specify which rows of a table should be updated. When used with the `UPDATE` statement, it restricts the update operation to only those rows that meet the condition defined in the `WHERE` clause. Without the `WHERE` clause, all rows in the table will be updated.
- `UPDATE EMPLOYEE SET salary = salary * 1.10 WHERE name = 'Bob';`

Create an index file for Lname attribute of the EMPLOYEE Table of a Database.

- An index in SQL is a database object used to improve the speed of data retrieval operations. To create an index on the `Lname` attribute of the `EMPLOYEE` table, you can use the `CREATE INDEX` statement.
- **SQL Syntax:** `CREATE INDEX index_name ON EMPLOYEE(Lname);`
- Example : `CREATE INDEX idx_lname ON EMPLOYEE(Lname);`
-

What is the difference between static and dynamic hashing?

- Both **static** and **dynamic hashing** are techniques used in database systems to manage hash tables. The main difference lies in how the size of the hash table is handled during insertion and deletion operations.
- **Static Hashing**:
  - In static hashing, the size of the hash table is fixed and does not change.
  - It uses a fixed number of buckets (or slots) to store data.
  - If the table becomes full or experiences a high number of collisions, performance degrades because the fixed number of buckets cannot accommodate more entries without a performance hit.
  - There is no automatic resizing of the hash table, making static hashing less flexible.
- **Dynamic Hashing**:
  - In dynamic hashing, the size of the hash table can grow or shrink dynamically as more data is added or removed.
  - When the number of records exceeds the current capacity of the hash table, the table can be expanded, and new buckets are created to accommodate the increased data.
  - Dynamic hashing helps maintain good performance even with a large amount of data.
  - It can handle growing datasets efficiently without significant degradation in query performance.
- **Key Difference**: Static hashing has a fixed size, while dynamic hashing allows the hash table to adjust its size during runtime, ensuring better handling of large and growing

datasets.

What is a dense index and a sparse index?

- Both **dense** and **sparse** indexes are techniques used to speed up data retrieval in database systems. They differ primarily in the number of entries they contain and how they relate to the data table.
- **Dense Index :**
  - A **dense index** is an index in which **every record** (or row) in the data table has a corresponding entry in the index.
  - **Structure**: In a dense index, for each record in the table, there is an index entry. This means that the index contains a key for every row in the table.
  - **Space Usage**: Since it indexes every record, it uses more storage space. The index is larger as it has one entry for each row in the data table.
  - **Lookup Performance**: Dense indexes allow **faster lookups** because each row in the data table can be directly mapped to an index entry.
  - **Example**: If a table has 100 rows, a dense index will have 100 index entries, each corresponding to a row in the table.
  - **Example Scenario**:
    - Consider an employee table where each employee has a unique employee ID. A dense index would contain an entry for every employee (every employee ID) and point to the actual record in the table.
- **Sparse Index :**
  - A **sparse index**, on the other hand, **does not** contain an index entry for every record. Instead, it contains an entry for only **some of the rows**, often one per data block or page in the table.
  - **Structure**: A sparse index typically contains entries only for the first record in each data block or page. It might not have an index entry for every row in the table, but it will have enough entries to allow for efficient searching.
  - **Space Usage**: Sparse indexes are **more space-efficient** because they don't store an entry for every row. They require less storage compared to dense indexes.
  - **Lookup Performance**: Sparse indexes can be slower to look up compared to dense indexes because the system may need to read more data or check multiple blocks to find the required entry.
  - **Example**: If a table has 100 rows and data is stored in blocks of 10 rows, a sparse index would contain only 10 entries, one for each block (or page).
  - **Example Scenario**:
    - Consider a table with employee data, where employee names are indexed. Instead of indexing every employee name, the sparse index might index only the

first name in each block of 10 records.

1 Marks

What is the difference between DELETE, TRUNCATE, and DROP?

- **DELETE**:
  - Removes rows from a table based on a condition (with a WHERE clause).
  - It can be rolled back (if using transactions).
  - It does not affect the table structure or schema.
- **TRUNCATE**:
  - Removes all rows from a table without any condition.
  - It is faster than DELETE because it does not log individual row deletions.
  - Cannot be rolled back in some DBMS (depending on transaction settings).
  - Does not affect the table structure or schema.
- **DROP**:
  - Removes the entire table, including its data, structure, and schema.
  - Cannot be rolled back.
  - The table and its data are permanently deleted.

Define data integrity.

- **Data integrity** refers to the accuracy, consistency, and reliability of data throughout its lifecycle. It ensures that data is correct, complete, and stored in a manner that prevents unauthorized access, corruption, or loss. Data integrity is maintained through validation rules, constraints, and regular data audits.

How do you enforce referential integrity in a database?

- **Referential integrity** is enforced using **foreign keys**. A foreign key ensures that a value in one table matches a value in another table. It can be maintained with cascading actions like **CASCADE**, **SET NULL**, **NO ACTION**, or **RESTRICT** to ensure consistency between related tables.

Define Conflict serializability.

- **Conflict serializability** refers to the ability to rearrange the operations (reads and writes) of transactions in a way that results in the same final database state as a serial execution, without violating any data consistency. It ensures that transactions can be reordered without conflicts, preserving the correctness of the database.

Write a common difference between logical and physical query optimization.

- A common difference between **logical** and **physical query optimization** is:
  - **Logical query optimization** focuses on choosing the most efficient **logical** execution plan for a query (e.g., reordering joins, pushing selections), independent of how the data is stored.
  - **Physical query optimization** focuses on selecting the most efficient **physical** plan (e.g., choosing indexes, join methods) based on the actual data storage and access methods.

The duplication of Data is called.....

- Data redundancy

In PL/SQL. what is meaning of P?

- Parameter

How BETWEEN operator work?

- The **BETWEEN** operator in SQL is used to filter the result set within a specified range. It can be used for **numeric**, **date**, or **text** values. It includes the **boundary values** in the range.
- Example :
  - `SELECT * FROM tableName WHERE column BETWEEN value1 AND value2;`

Write syntax for IN operator.

- ```
  WHERE department IN ('Sales', 'Marketing', 'HR');```

Why is file organization important in database management systems?

- **File organization** in DBMS is important because it affects data access speed, storage efficiency, data integrity, and system performance. Proper organization ensures fast retrieval, optimal space usage, and effective management of large datasets.

What is a record?

- A **record** is a collection of related data items (fields or attributes) stored as a single unit in a database, typically representing a single entity or object. Each field in a record contains a specific piece of information about that entity.

Define overflow in hashing.

- **Overflow in hashing** occurs when more data is inserted into a hash table than it can accommodate, causing multiple elements to hash to the same index. This results in a need for handling collisions, often using techniques like **chaining** or **open addressing**.

What is a hash function?

- A **hash function** is a function that takes an input (or key) and returns a fixed-size string of bytes, typically a hash code, which is used to index data in a hash table. It helps in quickly locating a data record based on its key.

Differentiate b/w DBMS & OODBMS.

- **DBMS (Database Management System)**:
  - Stores data in tables (relational model).
  - Uses SQL for querying.
  - Data is organized in rows and columns.
- **OODBMS (Object-Oriented Database Management System)**:
  - Stores data as objects (object-oriented model).
  - Supports concepts like inheritance, polymorphism, and encapsulation.
  - Data is organized as objects, similar to how data is represented in programming languages.